Лабораторна робота №6

Об'єктно орієнтоване програмування

Мета роботи: опанувати на практиці основні парадигми ООП за допомогою мови Python.

Короткі теоретичні відомості

Об'єктно-орієнтоване програмування (ООП) в мові Python має певні особливості. Наприклад, модифікатори доступу у Python, хоча і існують, але скоріше виступають конвенціями, ніж жорсткими правилами.

Множинне успадкування є іншою важливою особливістю ООП в Python. Це дозволяє класам успадковувати властивості та методи від кількох батьківських класів, що забезпечує більшу гнучкість у використанні та організації коду. Однак, слід відзначити, що ця гнучкість може вести до складніших структур, тому важливо використовувати її з обережністю.

Крім того, у Python всі методи є віртуальними. У викликах методів використовується ключове слово self, яке вказує на поточний об'єкт класу. Класи в Python також є об'єктами, що відкриває можливості для метапрограмування та динамічної зміни їхньої структури під час виконання програми. Це може бути використано для створення дуже абстрактних та гнучких архітектур програм.

Python підтримує усі основні принципи ООП: роботу з класами та об'єктами, інкапсуляцію, успадкування та поліморфізм. Класи служать шаблонами для створення об'єктів, об'єднуючи дані та функції в єдиному блоку коду. Інкапсуляція дозволяє об'єднувати дані та методи у єдиному об'єкті, приховуючи деталі реалізації і надаючи повний контроль над тим, як із об’єктом можна взаємодіяти. Успадкування дозволяє створювати нові класи на основі вже існуючих, що полегшує розширення та використання коду. Поліморфізм дозволяє використовувати об'єкти різних класів через спільний інтерфейс.

Завдання

В цій лабораторній роботі вам пропонується реалізувати деяку модель. Уся лабораторна робота – це одне завдання, за яке сумарно можна отримати до 16 балів.

Виконання основного завдання: 10 балів.

UML-діаграма класів: 2 бали.

Додаткове завдання: 4 бали.

Додаткове завдання наведене в кінці варіанту і, насправді, легко реалізується, якщо ви правильно реалізували основну частину.

Опис технічного завдання місцями може бути неповним, отже за потреби ви можете додавати атрибути, методи, навіть класи, якщо це виправдано.

Опис не дає інструкцій, як візуалізувати те, що відбувається у вас в програмі – ви маєте подбати про це самі. Подумайте і вирішіть, як і що має друкуватись на екрані під час роботи вашого коду, але так, щоб було зрозуміло, що коїться у віртуальному всесвіті вашої програми.

Також, в цьому описі немає жодної згадки про модифікатори доступу. Вирішуйте самі, які атрибути і методи робити приватними, які – захищеними, а які – публічними.

Варіант 1.

В деякому навчальному закладі є професор і група студентів. В цьому завданні вам потрібно змоделювати проведення тестування в групах. Далі будуть описані класи, які ви маєте реалізувати.

Клас `Question`

Клас описує одне питання з тесту.

Атрибути:

`question_text: str` – питання,

`answer: int` – відповідь

Під питанням далі будуть матись на увазі рядки `str` типу `"34 + 67"` або `"12 * 657"`. Тобто питання містить два цілих числа, а між ними - знак арифметичної дії (`"+"`, `"-"`, `"*"`) (для простоти не беремо ділення, бо там і дробові відповіді, і ризик ділення на нуль).

Конструктор приймає один обов'язковий параметр - `question_text`, і опціональну відповідь `answer` (дефолтно має значення `None`).

Методи:

`get_question_text()` – гетер для тексту питання;

`take_answer()`, сетер для відповіді. Може записати відповідь тільки один раз (тобто виправити відповідь не можна).

Клас `Test`

Клас описує тест.

Атрибути:

`student_name` - ім'я студента, який пише цей тест, дефолтно `None`.

`question_list` - список питань (об'єктів класу `Question`).

`mark` - оцінка, дефолтно `None`.

Методи:

Конструктор: створює порожній список питань, ініціалізує ім'я і оцінку значенням `None`.

`add_question(question: Question)` – додає питання до списку питань.

`set_name(name: str)` - приймає ім'я студента і записує його в поле `student_name`.

`__str__()` – повертає текстову репрезентацію поточного стану тесту: ім'я студента, список питань із відповідями, оцінку.

`__len__()` – повертає кількість питань у тесті.

`__bool__()` – повертає `True`, якщо на всі питання в тесті надані відповіді.

Клас `Person`

Клас описує особу. Від цього класу успадковуються класи `Student` і `Professor`.

Атрибути:

`name`

`age`

Методи:

`__str__()` – друкує ім’я і вік особи;

`get_name()` – гетер для імені;

`get_age()` – гетер для віку;

Конструктор приймає ім’я особи.

Клас `Student` (успадковується від класу `Person`).

Атрибути:

`knowledge` (`float` число від 0 до 1) – ймовірність дати правильну відповідь.

Методи:

`answer_question(question: Question)`, який приймає об'єкт `question` типу `Question` і заповнює його поле `answer` (через метод `take_answer` класу `Question`). Причому студент дає правильну відповідь із імовірністю `knowledge`. Тобто студент реально дивиться завдання (поле `question_text` питання), аналізує його (парсить) і обчислює результат, записуючи відповідь у питання через сетер `take_answer` класу `Question`.

`write_test(test: Test)`, який приймає об'єкт типу `Test`, проходиться по питаннях і відповідає на усі питання.

`get_description()` – повертає слово "smart"/"average"/"dumb" за таким правилом: smart (knowledge >= 0.8), average (0.5 <= knowledge < 0.8), dumb (knowledge < 0.5)

`__str__()` - друкує студента у форматі name - age - description.

Клас `Group`

Атрибути:

`student_list` - список студентів, який ініціалізується конструктором як порожній.

Методи:

`add_student(student: Student)` - додає студента до списку

`generate_group(number_of_students: int)` - створює рандомних студентів (з рандомними іменами і рандомним рівнем_знань)

`write_test(test: Test)` - уся група пише тест, причому метод приймає один об'єкт класу `Test` і кожному студенту видає копію (яку треба зробити за допомогою методу `deepcopy`, який імпортується з модуля `copy`), студенти пишуть свої тести, а метод `write_test` повертає список тестів, які написали учні.

`print_group_list()` - друкує список групи.

Клас Professor (успадковується від класу `Person`).

Методи:

`create_question()` - генерує рандомне питання (об'єкт типу `Question`)

`create_test(number_of_questions)` - генерує тест, використовує метод `create_question()` для створення питань. Паралельно він створює ще одну копію тесту з заповненими відповідями, які йому знадобляться для перевірки робіт. Професор ніколи не помиляється. Цей метод має повернути тест без відповідей і тест із відповідями як кортеж з двох тестів.

`check_test(test: Test, test_with_answers: Test)` - перевіряє окремий тест `test`, зіставляючи відповіді у `test` і `test_with_answers`, який він згенерував методом `create_test`. Тобто під час виклику методу `check_test`, аргументами треба передати 2 тести: тест, який перевіряється і тест із правильними відповідями. Результатом буде оцінка у тесті (поле `mark`) класу `Test`. Як він ставить оцінку – придумайте самостійно.

`check_group(test_list: list, test_with_answers: Test)` - перевіряє групу. Приймає список тестів, проходиться по них, виставляє в них оцінки. Другий параметр - тест із правильними відповідями.

`publish_results(test_list: list)` - друкує таблицю з двох колонок: `student_name` і `mark`.

Після опису цих класів, в окремому файлі імпортувати усі класи, створити групу з 8 студентів, надрукувати список групи, створити професора, нехай професор придумає тест з 5 питань, проведе контрольну в групі, перевірить студентські роботі і оголосить результати. Також вивести на екран найкращу і найгіршу роботи студентів.

<u>Додаткове завдання:</u> реалізуйте додаткову можливість користувачу зіграти за одного зі студентів. Тобто, коли цьому студенту настає час писати

тест, ці питання виводяться на екран, і він має відповідати на них. Ця модифікація має бути такою, що її можна відключити і запустити симуляцію без участі користувача.

<u>Примітка</u>: не використовувати функцію `eval`, модуль `re`. Якщо ви вважаєте, що вам потрібні додаткові атрибути чи методи класів, то реалізуйте їх. Подумайте, які модифікатори доступу варто зазначати для атрибутів і методів ваших класів.

Варіант 2.

В майбутньому стали популярними віртуальні космічні бої. Сутність такого змагання полягає в тому, що два віртуальних космічних кораблі ведуть бій на знищення.

Кожен корабель має **дві гарматні системи** і **генератор захисного поля**.

Гарматні системи можуть бути **лазерними** чи **кінетичними** (тобто корабель може мати 2 лазерні, 2 кінетичні або лазерну і кінетичну пушки). **Лазерна пушка** має відносно невелику руйнівну силу, але здатна проникати через захисні силові поля з деяким невеликим ослабленням (поле може поглинути до 10% ушкодження). **Кінетична пушка** — це система, яка за допомогою електромагнітного поля розганяє снаряд до надзвичайно високих швидкостей. Якщо такий снаряд досягає цілі, він наносить катастрофічні ушкодження, проте він може майже повністю бути блокований справним захисним силовим полем корабля (поле може поглинути від 70 до 95% ушкодження).

Захисне силове поле може одночасно захищати одну з трьох систем корабля: **одну з двох гарматних систем або генератор поля**.

Будь-яка гарматна система або генератор поля має свій стан цілісності (неушкодженості) – число від 0 до 30, де 0 — повністю зруйнований, а 30 — абсолютно неушкоджений. Якщо система повністю зруйнована, вона не може виконувати свої функції: пушка не стріляє, генератор поля не захищає. Корпус має власний стан цілісності — число від 0 до 100.

Коли ворожий корабель наносить удар, він цілиться в одну із систем корабля: одну з гармат (першу чи другу) або генератор захисного поля. При успішному влучанні страждає система і корпус корабля в рівному степені. Якщо влучання було в систему, яка вже зруйнована, то ушкодження отримує тільки корпус.

Бій відбувається таким чином:

1. Один з кораблів обирає, куди буде наносити удар (перша гармата, друга гармата чи генератор поля супротивника), а також за допомогою якої власної гарматної системи він буде атакувати (якщо одна з гармат не працює, то атака здійснюється іншою системою; якщо уражені обидві системи, то корабель не атакує).
2. Другий корабель не знає, куди він буде атакований і обирає, яку систему він буде захищати генератором поля.
3. Відбувається атака першого корабля на другий, при цьому відбувається розрахунок ушкодження з урахуванням параметрів зброї, роботи захисного поля тощо.
4. Потім другий корабель атакує перший за аналогічним сценарієм.

Корабель вважається знищеним, якщо його корпус має нульову цілісність.

Далі йде опис класів, які треба реалізувати.

Клас `Equipment`

Це клас, який описує обладнання (пушки і генератори поля) і реалізує логіку цілісності і виходу з ладу. Від цього класу успадковуються класи `Shield` і `Gun`.

Атрибути:

`hp` – цілісність обладнання (пушки чи генератора поля), дефолтне значення задається конструктором на рівні 30.

Методи:

`take_damage(damage: int)` – отримати ушкодження. Значення `hp` зменшується на `damage`, але не може стати від'ємним.

`__bool__()` – повертає `True`, якщо система працездатна (її `hp > 0`).

Клас `Gun`

Цей клас описує екземпляри гармат. Конструктор приймає тільки тип гармати, а всі інші параметри генеруються випадково у заданих межах, які описані нижче.

Атрибути:

`damage: int` – величина ушкоджень, яку може нанести ця гармата. Для лазерної зброї це 5-15 одиниць, а для кінетичної – 15-35. Таким чином, кращий зразок лазерної зброї наносить стільки ж ушкоджень, що і найгірший зразок кінетичної.

`min_accuracy: float` – мінімальна точність конкретного екземпляра, величина від 0.5 до 0.95. Коли відбувається постріл, від відбувається з точністю від `min_accuracy` до 1 (значення обирається випадково).

`gun_type: str` – тип гармати (`"laser"`, або `"kinetic"`);

`rating: str` – характеристика екземпляру зброї, яка показує, наскільки хорошим вийшов даний екземпляр. Має значення від `"*"` (жахливий) до `"*****"` (легендарний). На рейтинг впливає `damage` і `min_accuracy`. Наприклад, якщо в нас лазерна пушка з `damage = 6` і `min_accuracy = 0.52`, то це явно поганий зразок, має отримати рейтинг, `"*"`. А от кінетична гармата з `damage = 33` і `min_accuracy = 0.93` – це дуже крутий екземпляр, однозначно `"*****"`. Придумайте, яким чином обчислюється рейтинг.

Методи:

Конструктор приймає тип гармати, а значення `damage` і `min_accuracy` генеруються випадковим чином.

`calculate_damage()` – обчислює ушкодження з одиничного пострілу. Тут мають урахуватись параметри зброї.

Клас `Shield`

Методи:

`reduce_damage(damage, damage_type)` – приймає величину ушкоджень, нанесену ворожою гарматною системою, і тип ушкодження (лазерне чи кінетичне). Обчислює, який рівень ушкоджень проходить скрізь захисне поле. Наприклад, якщо атака була кінетичною, а її величина – 20, і поле блокувало 90%, то скрізь поле пройде лише 2 одиниці. Метод має повертати ушкодження, яке залишається після дії захисного поля, тобто 2.

Клас `Spaceship`

Описує бойовий космічний корабель. Корабель має корпус і три слоти для обладнання, в які за правилами гри встановлюється 2 пушки будь-якого типу і 1 генератор силового поля.

Атрибути:

`ship_name` – ім'я корабля;

`hp: int` – цілісність корпусу, дефолтне значення задається конструктором на рівні 100;

`equipment: list` – список систем корабля. Елементами цього списку мають бути об'єкти класів `Shield` або `Gun`;

`equipment_index_to_be_protected` – індекс у списку власного обладнання (`equipment`), який вказує на обладнання, яке корабель планує захищати полем у наступному етапі битви;

`equipment_index_to_be_attacked` – індекс у списку ворожого обладнання (`equipment`), який вказує на обладнання, яке корабель планує атакувати у наступному етапі битви;

`gun_to_attack_with` – індекс у списку власного обладнання (`equipment`), який вказує на гармату, якою корабель планує атакувати у наступному етапі битви;

Конструктор приймає ім'я корабля. Обладнання він створює випадковим чином.

Методи:

`get_equipment_config()` – метод, який повертає конфігурацію корабля. Наприклад, якщо в кораблі встановлено у першому слоті кінетичну пушку ("K" - kinetic), в другому – лазерну ("L" - laser), а в третьому – генератор поля ("S" - shield), то метод має повернути рядок "`KLS`", якщо ж обидві пушки – кінетичні, то отримаємо "`KKS`". Положення літери в цьому рядку відповідає індексу відповідного обладнання у списку equipment.

`prepare_to_attack(another_ship: Spaceship)` – корабель обирає, куди і чим він буде стріляти. Результатом роботи цього методу має бути оновлення значень `equipment_index_to_be_attacked` і `gun_to_attack_with`.

`prepare_to_defend()` – корабель обирає, яке обладнання він буде захищати. Результатом роботи цього методу має бути оновлення значення `equipment_index_to_be_protected`.

`take_damage(damage, target)` – метод, який реалізує отримання кораблем ушкоджень. Тут `damage` – це вхідний рівень ушкоджень. Якщо власний генератор поля захищає обладнання, в яке йде постріл (визначається параметром `target`), то ушкодження зменшуються (метод `reduce_damage` класу `Shield`).

`attack(another_ship: Spaceship)` – реалізація атаки даним кораблем іншого корабля. В цьому методі використовуються значення `equipment_index_to_be_attacked` і `gun_to_attack_with`, щоб отримати потужність пострілу і ціль пострілу. Потужність – це число, а ціль – це індекс у ворожому списку обладнання, куди йде атака. Ці дані передаються аргументами у метод `take_damage` ворожого корабля, а вже він обчислює наслідки.

`__bool__()` повертає `True`, якщо корпус корабля має `hp > 0`.

`__str__()` дає опис корабля і його систем у вигляді тексту. Не забудьте про рейтинг (зірочки), коли будете виводити інформацію про зброю.

Після того, як ви реалізуєте усі класи, створіть два кораблі. Змоделюйте один бій між кораблями. Нехай в процесі битви ваш код пише, що відбувається, який корабель атакує, чим саме атакує і якими були результати атаки.

Додаткове завдання: реалізуйте додаткову можливість користувачу зіграти за один з кораблів. Ця модифікація має бути такою, що її можна відключити і запустити симуляцію без участі користувача.

Примітка: якщо ви вважаєте, що вам потрібні додаткові атрибути чи методи класів, то реалізуйте їх.

Варіант 3.

В цьому завданні треба змоделювати життя в невеличкому світі, де усі мешканці грають в казино і користуються послугами банку. Казино дає змогу виграти гроші. Банк містить облікові записи клієнтів. У банку клієнти можуть: вносити кошти на рахунок, знімати кошти, користуватись кредитними коштами. В свою чергу, банк наприкінці кожного року нараховує відсоток на рахунок і контролює стан рахунків. Усе життя йде часовими кроками довжиною в місяць. За один місяць відбуваються такі події:

1. Мешканці оцінюють своє фінансове становище (суми на руках і рахунках), кладуть або знімають гроші з рахунків.
2. Мешканці йдуть в казино і грають.
3. Банк аналізує стан рахунків мешканців і за певними правилами блокує рахунки, нараховує відсотки на залишок на основних рахунках.

Нижче йде опис класів, які треба реалізувати.

Клас `World`

Цей клас описує невеличкий вигаданий замкнений світ, в якому є мешканці, банк і казино.

Атрибути:

`people: list` – список мешканців (об'єктів класу `Person`)

`bank: Bank`

`casino: Casino`

Методи:

Конструктор приймає єдиний параметр population – кількість населення у світі. Для кожного мешканця конструктор створює акаунт в банку.

`live()` – змушує світ жити протягом одного місяця, тобто відбуваються усі процеси: мешканці роблять щось із своїми рахунками, ідуть в казино, грають, а банк моніторить рахунки, нараховує відсотки.

Клас `BankAccount`

Атрибути:

`name: str`

`card_number: str` – номер картки у форматі "`xxxx xxxx xxxx xxxx`" (де `x` – цифра), генерується конструктором автоматично

`balance: float` – баланс на рахунку (дефолтне значення 0.0)

`currency: str` - валюта

`credit_limit: float` – кредитний ліміт, (дефолтне значення 10000.0)

`credit_funds: float` – кредитні кошти, дефолтне значення дорівнює `credit_limit`.

`interest_rate: float` – місячна відсоткова ставка (дефолтне значення – 0.02, тобто 2%);

`credit_has_not_been_repaid_for: int` – лічильник часу, який відслідковує, скільки місяців не повертались кредитні кошти;

`is_blocked: bool` – прапорець блокування акаунту.

Методи:

`deposit(funds: float)` - покласти на рахунок. Якщо до цього були використані кредитні кошти (тобто `credit_funds` < `credit_limit`), то спочатку поповнюється кредитний рахунок, а лише потім починає поповнюватись основний рахунок.

`withdraw(funds: float)` - зняти гроші з рахунку. Якщо на основному рахунку грошей недостатньо, то використовується кредитний рахунок, причому метод повертає повідомлення про те, що використані кошти кредитного рахунку. Якщо все одно не вистачає, то змін не відбувається, а метод повертає повідомлення про нестачу коштів.

`change_limit(funds: float)` - змінити кредитний ліміт. Це можна зробити тільки якщо у користувача немає кредитної заборгованості. Якщо йде спроба змінити ліміт, але при цьому є кредитна заборгованість, то ліміт не змінюється, а повертається відповідне повідомлення.

`__str__()` – повертає текстову репрезентацію поточного стану рахунку

`interest_accrual()` – здійснює нарахування відсотків. Нарахування здійснюється тільки за умови відсутності кредитної заборгованості і тільки на основні кошти (balance).

`credit_observe()` – спостерігає за кредитом: якщо він повернутий, записує в атрибут `credit_has_not_been_repaid_for` нуль (тобто скидає термін заборгованості), а якщо кредитні кошти використовуються, то збільшує цей параметр на 1 (мається на увазі, що банк буде викликати цей метод раз на місяць, щоб слідкувати за станом кредитів).

Конструктор приймає параметри `name` (обов'язково), `cash` (опціонально).

Клас `Casino`

Цей клас реалізує гру в казино на гроші. Грають в кості за такими правилами. Одна гра – це кидок двох кубиків. Якщо випадає дубль (однакові числа на кубиках), то ставка подвоюється, якщо випадає сума 10 чи 11, то ставка потроюється. Відповідно, якщо випало [∴][∴] (і дубль, і сума 10), то ставка збільшується в 6 разів. Якщо випала сума очок 6 або 7, то ставка просто зберігається. В усіх інших випадках ставка втрачається.

Методи:

`play(bet: float)` – гра в казино. Метод повертає виграш згідно з правилами, описаними вище. Таким чином, щоб коректно опрацювати одну гру, від грошей на руках людини треба відняти ставку і додати те, що поверне цей метод.

Клас `Bank`

Цей клас описує банк і логіку його роботи. Банк містить список акаунтів (рахунків) клієнтів. Кожного місяця банк нараховує відсоток на залишок на основному (некредитному) рахунку. Якщо клієнт має заборгованість довше, ніж 6 місяців (тобто кредитні кошти не повернуті), то акаунт блокується.

Атрибути:

`account_list: list` – список акаунтів

Методи:

`create_account(person: Person)` – створює акаунт для особи person, в поле account класу Person записує посилання на її акаунт.

`process_accounts()` – обробляє усі акаунти. Проходить по акаунтах, викликає метод `credit_observe()`. Потім якісь акаунти блокує, на якісь нараховує відсотки.

Клас `Person`

Атрибути:

`name: str`

`cash: float` – сума грошей на руках, генерується конструктором випадковим чином в певних межах.

`account: BankAccount` – посилання на власний акаунт в банку.

додаткові атрибути, які описують поведінку людини, задаються конструктором випадково, див. нижче

Методи:

`deposit(funds: float)` – цей метод реалізує процес поповнення рахунку. Він викликає однойменний метод класу `BankAccount` і забезпечує транзакцію (тобто сума має віднятись від атрибуту cash) і додатись до рахунку. Причому це має статись тільки якщо на руках достатньо коштів.

`withdraw(funds: float)` – цей метод реалізує процес зняття грошей з рахунку. Він викликає однойменний метод класу `BankAccount` і забезпечує транзакцію (тобто сума має віднятись від рахунку) і додатись до атрибуту cash тільки якщо операція з рахунком була вдалою.

`interact_with_bank()` – метод, в якому реалізовано логіку особи щодо операцій із власним рахунком. Вона може покласти кошти на рахунок, зняти кошти з рахунку, взяти кредитні кошти, погасити кредит. Логіку прийняття рішень придумайте і реалізуйте самостійно. Це має бути набір правил на кшталт «якщо на руках багато грошей, то треба їх покласти на рахунок, залишивши тільки суму, потрібну для гри в казино», «якщо на руках мало грошей, а на рахунку багато – то треба зняти гроші в кількості, потрібній для гри в казино в цьому місяці», «якщо висить кредит, то його треба погасити, якщо є чим», «якщо є кредит і його не треба терміново повертати, то можна зняти ще грошей, щоб відігратись в казино», і подібні. Ви можете додати потрібні атрибути до класу Person, які б описували характер людини: чи буде

вона ризикувати і лізти в кредит, чи просто зменшить свої ставки; де саме знаходяться оті межі «багато» і «мало», які ставки вона робить тощо.

`interact_with_casino(casino: Casino)` – метод, який реалізує гру в казино. Людина ставить гроші з тих, що в неї на руках, і отримує виграш на руки згідно з правилами гри. Вона не може поставити більше, ніж має на руках (атрибут `cash`).

Після того, як ви реалізуєте усі класи, створіть світ на 10 людей. Нехай світ почне жити за правилами, описаними в цьому завданні. Нехай на кожен місяць на екран коротко виводиться інформація про те, що сталось із кожним мешканцем у форматі: «ім'я; скільки взяв з рахунку/поклав на рахунок; скільки виграв/програв; грошей на руках; грошей на основному рахунку; кредит; статус рахунку». Змоделювати життя цього світу протягом деякого часу.

Додаткове завдання: реалізуйте додаткову можливість користувачу зіграти за одного з мешканців. Тобто коли код доходить до опрацювання дій цього мешканця, вибір надається користувачу, а не генератору випадкових чисел. Ця модифікація має бути такою, що її можна відключити і запустити симуляцію без участі користувача.

Примітка: якщо ви вважаєте, що вам потрібні додаткові атрибути чи методи класів, то реалізуйте їх.

Варіант 4

В майбутньому дуже популярними стали турніри, на яких б'ються людиноподібні велетенські роботи.

Робот може бити в голову, корпус чи ноги супротивника, а також захищати свою голову, корпус або ноги (щось одне). Бій включає такі етапи:

1. Робот А обирає, яку частину свого роботичного тіла він буде захищати.
2. Інший робот В обирає, куди він буде бити робота А.
3. Наноситься удар. Якщо робот А захищав саме ту частину свого тіла, куди прийшовся удар, то ушкоджень він не отримує. Натомість, якщо удар був нанесений в незахищену частину тіла, то він отримує ушкодження.
4. Далі все повторюється, тільки роботи міняються ролями.

Бій продовжується до тих пір, поки обидва роботи залишаються дієздатними. Кожен робот має свій запас міцності (шкала hp від 0 до 100).

Після бою робот відновлюється до початкового стану.

Турніри з таких боїв проводяться за ***олімпійською схемою***. На початку кількість учасників має бути степенем двійки (2, 4, 8, 16, 32 тощо). Учасники розбиваються на пари, і в кожній парі відбувається бій. Переможці переходять в наступний тур, а переможені вибувають з турніру. В наступному турі учасників вже вдвічі менше і все повторюється знову. Тури проводяться до тих пір, поки не залишиться один головний переможець.

В цій лабораторній роботі вам необхідно змоделювати проведення такого турніру.

Нижче йде опис класів, які треба реалізувати.

Enum-клас `BodyPart`

Це enum-клас, який описує частини тіла. Для створення enum-класів скористайтесь модулем enum, звідки можна імпортувати класи `Enum` або `IntEnum` і від них успадкувати свій клас `BodyPart`. Використання enum-класів робить код значно більш прозорим і зрозумілим. Цей enum-клас має містити такі значення:

```
HEAD, BODY, LEGS
```

Клас Punch

Цей маленький клас описує один удар. Він не має методів, окрім конструктора, але містить інформацію про силу удару і частину тіла, в яку цей удар був спрямований, а також про те, чи був цей удар критичним.

Атрибути:

`damage: int`

`bodypart: BodyPart`

`is_critical: bool` – прапорець, що сигналізує про критичний удар.

Методи:

Конструктор приймає значення `damage` і `bodypart`, і записує їх у відповідні атрибути.

Клас `Robot`

Атрибути:

`name: str` – ім'я робота. Якщо конструктор не отримує імені аргументом, то він генерує ім'я випадковим чином. Придумайте цікавий спосіб автоматичного неймінгу роботів. `"Dg46fh5"` – не дуже цікаве ім'я, а от `"Bloody_Crusher"` або `"Son_of_R2D2"` – вже краще.

`hp: int` – цілісність робота (шкала від 0 до 100, де 0 – робота виведено з ладу, а 100 – робот повністю справний)

`bodypart_to_hit: BodyPart` – частина тіла супротивника, яку робот планує атакувати;

`bodypart_to_protect: BodyPart` – частина власного тіла, яку робот буде захищати.

`critical_hit_multiplier: float` – це множник, на який множиться базова шкода від удару. Наприклад, якщо `critical_hit_multiplier` дорівнює 1.5, то це означає, що якщо удар вийшов критичним, то він нанесе 150% від стандартного урону. Параметр `critical_hit_multiplier` має варіюватись в діапазоні 1.2 – 1.8 і генерується конструктором випадково.

`critical_hit_chance: float` – ймовірність нанести критичний удар. Ця величина має варіюватись в діапазоні від 0.01 до 0.12 (від 1% до 12%) і генеруватись конструктором випадково.

Методи:

`prepare_to_defend()` – метод, в якому робот обирає, яку частину власного тіла він буде захищати. Цей метод змінює значення атрибуту `bodypart_to_protect`.

`prepare_to_attack()` – метод, в якому робот обирає, яку частину тіла супротивника він буде атакувати. Цей метод змінює значення атрибуту `bodypart_to_hit`.

`take_punch(punch: Punch)` – метод, який обчислює власні ушкодження. Він приймає об'єкт класу `Punch`, звідки отримує величину шкоди, яку наніс супротивник, аналізує, яку частину тіла захищено, куди було нанесено удар, і обчислює нове значення власного `hp`. Якщо було нанесено критичний удар (атрибут `is_critical` класу `Punch`), то навіть якщо робот захищає місце, куди удар був нанесений, то блокується тільки 50% шкоди.

`throw_punch()` – метод, який створює і повертає об'єкт удару класу `Punch`. Він має реалізувати плани робота щодо місця атаки. Сила атаки генерується випадковим чином в заданих межах в залежності від частини тіла, куди наносить удар. Різні типи ударів наносять різну кількість ушкоджень: удар в голову завдає 8-20 одиниць шкоди, удар в корпус – 7-12 одиниць шкоди,

а удар в ноги – 5-15 одиниць шкоди. Також треба врахувати ймовірність нанести критичний удар.

`__str__()` – повертає текстове подання стану роботу: ім'я – кількість `hp`.

`rating: str` – характеристика робота, яка показує, наскільки хорошим вийшов даний екземпляр. Має значення від "*" (жахливий) до "*****" (легендарний). На рейтинг впливає critical_hit_multiplier і critical_hit_chance. Наприклад, якщо в нас critical_hit_multiplier = 1.23 і critical_hit_chance = `0.012`, то це явно не дуже вдалий робот, який майже ніколи не наносить критичних ударів, а, навіть, коли йому вдається, цей критичний удар не набагато потужніший за звичайний, отже, він має отримати рейтинг, "*". А от якщо critical_hit_multiplier = 1.76 і critical_hit_chance = `0.113` – то це дуже крутий робот, який доволі часто наносить критичні удари, які, до того ж, дуже потужні, це рейтинг "*****". Придумайте, яким чином обчислювати такий рейтинг.

`detailed_description()` – повертає детальний опис робота.

`respawn()` – відновлює стан робота до неушкодженого

Клас `Match`

Цей клас описує одиничний поєдинок двох роботів.

Атрибути:

`robot1: Robot`

`robot2: Robot`

`winner: Robot | NoneType` – сюди має бути записаний переможець (об'єкт класу `Robot`), а до того значення має бути `None`.

`is_conducted` – булевий прапорець, який відповідає піднімається, коли матч проведено.

Методи:

`conduct()` – метод, який запускає проведення матчу.

`__str__()` – текстове подання опису матчу (які роботи б'ються).

`__bool__()` – повертає True, якщо матч відбувся

Конструктор приймає двох роботів, які мають битись в цьому матчі, ініціалізує атрибути winner, `is_conducted`.

Клас `Tournament`

Цей клас описує весь турнір.

Атрибути:

`robot_list: list` – список усіх роботів, що беруть участь у змаганнях

Методи:

`add_participant(robot: Robot)` – додає чергового робота до списку учасників;

`create_participants(number_of_participants)` – метод створює зазначену кількість учасників турніру;

`conduct()` – метод запускає і проводить весь турнір. Турнір проводиться тільки у випадку, якщо кількість учасників відповідає умовам олімпійської схеми (тобто є степенем двійки), в іншому випадку виводиться повідомлення про неможливість провести турнір.

Після того, як ви реалізуєте усі класи, створіть турнір, наповніть його гравцями і проведіть турнір. Під час проведення турніру програма має друкувати, що відбувається, хто з ким б'ється, які тури розігруються тощо.

Додаткове завдання: реалізуйте додаткову можливість користувачу зіграти за одного з роботів. Тобто коли доходить черга до бою з гравцем-людиною, вибір, куди бити і яку частину тіла захищати, надається користувачу, а не генератору випадкових чисел. Гравець не має контролю над тим, чи вийде удар критичним. Ця модифікація має бути такою, що її можна відключити і запустити симуляцію без участі користувача.

Примітка: якщо ви вважаєте, що вам потрібні додаткові атрибути чи методи класів, то реалізуйте їх.

Варіант 5.

Існує світ, який складається з клітчастого поля. В цьому світі живуть істоти, яких можна розділити на **хижаків** (Predator) і **здобич** (Prey). Хижаки полюють на здобич, причому ведуть самотній спосіб життя. Здобич значно слабша за хижака, але якщо хижак дуже ослаблений, а здобич сита і сильна, то вона може його подолати. Здобич харчується **травою**, а хижаки – **здобиччю**. З часом істота, яка не їла, слабшає.

Кожна істота має **здоров'я** (hp), **рівень голоду** (hunger_level) і **силу** (strength). З плином часу (тобто за кожен крок існування світу) голод сильнішає, і коли він досягає деякого **межового значення** (max_hunger_level), починає зменшуватись сила істоти, а значить, вона буде слабше атакувати супротивника. Коли сила істоти впала до половини, істота разом з силою починає втрачати здоров'я. Коли істота поїла, її сила відновлюється і рівень голоду падає до 0. Здоров'я відновлюється повільно саме по собі тільки тоді, коли істота не голодна. Якщо здоров'я падає до 0, істота вмирає. Труп істоти не залишається на карті, а просто зникає.

Хижак вміє відслідковувати здобич по **слідах**. Здобич, коли пересувається, залишає слід, який з часом слабшає і зникає. Хижак є безстрашним і завжди йде в напрямку посилення сліду здобичі. Хижак також залишає слід в клітинках, де він пройшов. Хижаки ігнорують сліди інших хижаків. Клітинка може мати лише один слід – слід останньої істоти, що там була. Якщо слід обривається, то істота далі рухається рандомно.

Здобич має дві мети: уникати хижаків і шукати траву. Якщо здобич натрапляє на слід хижака, то вона буде рухатись у напрямку послаблення сліду.

Коли істота не бачить слідів, вона рухається в довільному напрямку, сподіваючись знайти їжу. Коли знаходить їжу, вона її їсть.

Між істотами стається бійка на смерть, якщо вони опиняються в одній клітинці карти. Хижак не буде атакувати хижака, а здобич не чіпає здобич. Якщо хижак здолав здобич, він одразу її з'їдає.

В цій лабораторній роботі вам пропонується змоделювати існування такого світу.

Нижче йде опис класів, які треба реалізувати.

Клас `World`

Цей клас представляє увесь світ. Містить карту місцевості і список істот.

Атрибути:

`map: Map`

`creatures: list`

Методи:

Конструктор приймає розміри карти і кількості хижаків і здобичі, формує карту заданого розміру, заповнює її травою, створює і додає усіх істот до списку. Зверніть увагу, список істот містить і хижаків, і здобич.

`next_step()` – просуває життя світу на один крок уперед. Усі істоти діють відповідно до своєї поведінки (хижаки полюють, здобич уникає хижаків і шукає траву), сліди на карті слабшають, десь з'являється трава, істоти різного типу б'ються, якщо зустрічаються.

`check_encounters()` – перевіряє, чи не зустрілись якісь істоти між собою. Якщо зустрілись, і вони різного типу, то між ними відбувається бійка (метод conduct класу `Fight`). Якщо хижак здолав здобич, то він одразу її з'їдає.

`live()` – відтворює життя світу (викликає метод `next_step()` в циклі)

`display()` – виводить в консоль текстову репрезентацію світу. Має виводитись таблиця, у якій зображується наповнення мапи. Наприклад, вона може виглядати так:

```
.   .   .   .   .   .   .   .   .   .
.   G   1   .   .   1'  2'  3'  .   .
.   3   2   .   .   G   .   4'  .   .
.   4   .   .   .   .   .   5'  6'  .
6   5   G   .   .   .   .   .   7'  .
7   .   .   .   .   .  [X]  9'  8'  .
8   9  (P)  .   .   .   .   .   .   .
.   .   .   .   .   .   .   G   G   .
```

Тут `G` – grass, `6` – сила сліду здобичі, `6'` – сила сліду хижака, `[X]` – хижак, `(P)` – здобич. Візуалізацію можна втілити, як вам хочеться, але щоб вона була не гіршою за наведений приклад.

Клас `Map`

Цей клас містить мапу і забезпечує інтерфейс взаємодії з нею.

Атрибути:

`data` – двовимірний список, який в кожній своїй клітинці містить її статичне наповнення. За наповнення клітинки відповідає клас `Cell`. Елементи цього класу містять інформацію про клітинку: чи є там трава, і чи є там сліди. Мапа не містить істот, але кожна істота має атрибут-посилання на мапу для того, щоб мати змогу отримувати інформацію про оточення. Істоти безпосередньо не бачать одна одну на відстані, вони зіштовхуються тільки тоді, коли опиняються на одній клітинці. Контролем таких зустрічей займається метод `check_encounters()` класу `World`.

Методи:

Конструктор приймає розміри мапи і базову ймовірність трави у комірці.

`get_cell(x, y)` – повертає значення, яке міститься у комірці.

Клас Creature

Є абстрактним класом (не треба створювати об'єкти цього класу). Представляє загальний шаблон істоти. Від цього класу успадковуються класи Predator і Prey.

Атрибути:

`x: int` – координати істоти на карті

`y: int` – координати істоти на карті

`hp: int`

`initial_strength: int` – базова сила тварини, потрібна для того, щоб відновити втрачену силу, якщо істота спочатку ослабла, а потім поїла. Половина цієї величини визначає межу, після якої починається втрата здоров'я.

`strength: int` – поточна сила.

`hunger_level: int`

`max_hunger_level: int`

`__bool__()` – повертає `True`, якщо тварина жива (hp>0)

Методи:

`take_damage(damage: int)` – метод, який приймає значення шкоди і змінює кількість hp істоти

`eat()` – метод, який реалізує процес їжі. Скидається лічильник hunger_level, повертаються сили, їжа прибирається з мапи.

`move()` – абстрактний метод, який треба перевизначити у класах-нащадках. Цей метод відповідає за переміщення істоти. Істота може рухатись в будь-якому з 8 напрямків, але тільки на 1 клітинку.

`act()` – метод, який реалізує усі дії істоти: вона аналізує, чи не натрапила вона на слід, якщо натрапила, то на який. Далі істота обирає, як їй рухатись. Якщо це здобич і вона натрапила на траву, то вона їсть. Якщо істота не голодна, але має неповну шкалу здоров'я, то вона лікується на 3 одиниці за крок. В цьому класі цей метод має бути абстрактним, але в нащадках він має бути перевизначеним відповідно до субкласу.

Клас `Predator` (успадковується від класу `Creature`)

Перевизначає методи `move()`, `act()` і інші, які ви вважаєте за потрібне перевизначити відповідно до логіки класу `Predator`.

Клас `Prey` (успадковується від класу `Creature`)

Перевизначає методи `move()`, `act()` і інші, які ви вважаєте за потрібне перевизначити відповідно до логіки класу `Prey`.

Клас `Cell`

Клас, який відповідає за стан клітинки на мапі.

Атрибути:

`grass_present: bool` – прапорець, який відповідає за наявність у клітинці трави.

`trace_status: NoneType | str` – чи є в клітинці слід? Якщо сліду нема, то значення цього атрибуту – `None`. Якщо є – то використовується рядок `"Predator"` чи `"Prey"` в залежності від того, хто залишив слід. Клітинка може мати слід істоти і траву одночасно. Якщо істота-здобич тікає від хижака, але опинилась у клітинці з травою, вона все одно з'їсть траву.

`initial_trace_magnitude:int` – початкова сила сліду, дефолтне значення 10. Коли істота тільки опинилась у клітинці, вона створює слід силою `initial_trace_magnitude`, а потім, на кожному кроці існування світу, інтенсивність сліду падає на 1. Через `initial_trace_magnitude` кроків слід зникає повністю.

`trace_magnitude:int` – поточна сила сліду

`grass_birth_probability: float` – імовірність появи трави. Дефолтне значення 0.02 (тобто 2%).

Методи:

`fade_out()` – зменшує силу сліду (атрибут `trace_magnitude`) на 1. Якщо ця величина досягає значення 0, то слід зникає і `trace_status` отримує значення `None`.

`grow_grass()` – метод, який з імовірністю `grass_birth_probability` створює траву в клітинці, якщо її тут ще не було.

Клас `Fight`

Клас, який реалізує бій істот. Істоти по черзі наносять удари, сила яких дорівнює значенню параметру strength. Значення силу удару передається в метод `take_damage` класу `Creature`. Першою завжди атакує сильніша істота. Може статись так, що в одній клітинці зустрінеться хижак і дві здобичі. Тоді хижак атакує якусь здобич, а здобичі атакують хижака. Якщо здобичей декілька, то в них з'являється більше шансів вижити. Якщо у бій вступить кілька хижаків, то вони не будуть атакувати одне одного і, якщо хижаки переможуть, то усі будуть ситі.

Атрибути:

`creatures: list` – список тварин, що будуть битись.

`survivors: list` – список, який містить переможців (об'єкти класу `Predator` або `Prey`).

`is_conducted` – булевий прапорець, який піднімається, коли битва завершилася.

Методи:

`conduct()` – метод, який запускає проведення битви.

`__str__()` – текстове подання опису битви.

`__bool__()` – повертає `True`, якщо бійка відбулась.

Конструктор приймає довільну кількість істот, які мають битись на смерть, ініціалізує атрибути `survivors`, `is_conducted`.

Після того, як ви реалізуєте усі класи, створіть світ розміру 10x15, створіть у ньому хижаків і здобич, і запустить симуляцію. Кожен крок існування світу має візуалізуватись.

<u>Додаткове завдання:</u> реалізуйте графічну візуалізацію цього світу. Можна використовувати будь-яку графічну бібліотеку, наприклад, **`pygame`**.

Зміст звіту

1. Титульний лист із назвою лабораторної роботи, номером свого варіанту, ПІБ студента та групи.
2. Мета лабораторної роботи.
3. Завдання.
4. UML-діаграми класів.
5. Лістинг програми (скопійований з IDE).
6. Результати роботи програми (скріншоти).

Окремо в ментор прикріпити архів із файлами проєкту.